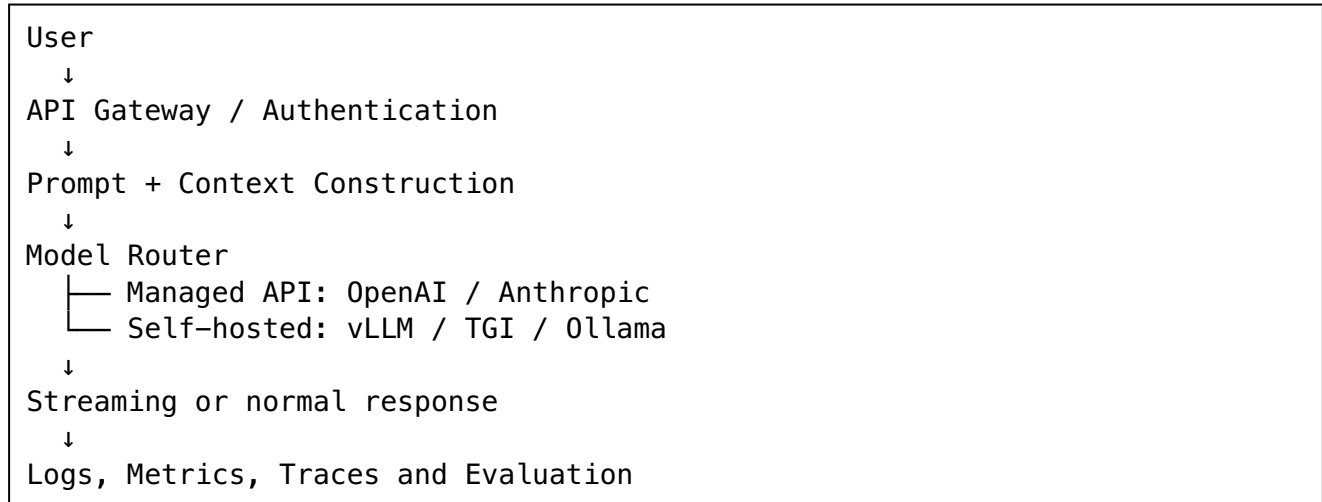


Day 17 – Inference, Deployment, LLMOps & Monitoring

1. The production LLM lifecycle

Training produces a model, but **inference** is where the model serves real user requests.

A production LLM system usually follows this path:



A Senior AI Engineer must optimize four dimensions:

1. **Quality** — Is the answer correct, grounded and safe?
2. **Latency** — How quickly does the user see a response?
3. **Throughput** — How many requests or tokens can the system process?
4. **Cost** — What is the cost per successful request?

A useful interview statement is:

LLM deployment is not only about serving a model. It is about serving a versioned model-prompt-retrieval system with measurable quality, latency, reliability and cost.

2. Inference options

2.1 API-based inference

In API-based inference, your application sends a request to a managed provider such as OpenAI or Anthropic.



The provider manages:

- Model hosting
- GPUs
- Scaling
- Model runtime

- Hardware failures
- Inference optimization
- API availability

Managed APIs generally expose token-usage information, which can be used for cost tracking and model-routing decisions. Streaming APIs can return generated output incrementally rather than waiting for the complete response. ([OpenAI Platform](#))

Advantages

- Fastest way to launch a product
- No GPU or inference-server operations
- Easy access to powerful models
- Usually provides SDKs, streaming and usage metadata
- Can switch models through configuration
- Suitable when traffic is unpredictable

Limitations

- Cost increases with token volume
- Provider rate limits may constrain throughput
- Network latency is outside your control
- Possible vendor dependency
- Less control over model internals and serving configuration
- Data residency and privacy requirements require careful review
- Provider model updates can alter behaviour

Best use cases

- Early-stage products and prototypes
- Low-to-medium request volume
- Applications requiring frontier-model quality
- Teams without an ML infrastructure function
- Workloads with bursty or unpredictable traffic

Production recommendation

Do not call a provider SDK throughout your business code. Create an abstraction:

```
class LLMProvider:
    def generate(self, request):
        raise NotImplementedError
```

Possible implementations:

```
OpenAIProvider
AnthropicProvider
VLLMProvider
TGIPProvider
```

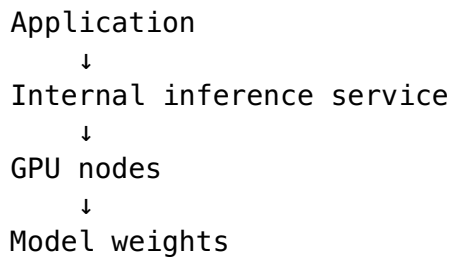
This makes it easier to add:

- Provider fallback
- Model routing

- Retries
 - Circuit breakers
 - Cost controls
 - A/B testing
 - Migration to self-hosting
-

2.2 Self-hosted inference

With self-hosting, your organization deploys the model weights and inference runtime on its own infrastructure.



You are responsible for:

- GPU provisioning
- Model loading
- Autoscaling
- Batching
- Monitoring
- Capacity planning
- Security patches
- High availability
- Model and runtime upgrades

Advantages

- More control over models and serving configuration
- Better data isolation
- Predictable infrastructure for stable, high-volume workloads
- Ability to use open-weight or fine-tuned models
- Control over quantization, batching and GPU allocation
- Lower marginal cost may be possible at sufficiently high utilization

Limitations

- Significant GPU and platform complexity
- Idle GPUs still cost money
- Capacity planning is difficult
- Model loading can be slow
- Scaling GPU workloads is slower than ordinary stateless services
- Requires specialized monitoring and operational knowledge

Best use cases

- Stable and high inference volume

- Strict privacy or data-residency requirements
 - Fine-tuned proprietary models
 - Offline or isolated environments
 - Workloads where a smaller specialized model performs sufficiently well
-

3. Common self-hosted inference engines

3.1 vLLM

vLLM is commonly used for high-throughput LLM serving. It provides an HTTP server compatible with several OpenAI-style APIs, making it easier to replace a managed API with a self-hosted endpoint.

([vLLM](#))

Important concepts include:

- Continuous batching
- Efficient KV-cache memory management
- Tensor parallelism
- Quantized model support
- Streaming
- OpenAI-compatible interfaces

When to propose vLLM

- Production serving of open-weight models
 - High concurrent request volume
 - GPU utilization is important
 - Existing applications already use OpenAI-compatible clients
-

3.2 Hugging Face TGI

Text Generation Inference, or TGI, is a production-oriented server for open-source language models.

Its documented features include:

- Continuous batching
- Token streaming through SSE
- Quantization
- Paged attention and KV caching
- Prometheus metrics
- OpenTelemetry tracing
- OpenAI-compatible API endpoints ([Hugging Face](#))

TGI uses a router to receive requests, create batches and communicate with one or more model-server shards. ([Hugging Face](#))

When to propose TGI

- Hugging Face-based model ecosystems
- Production GPU deployment
- Teams wanting built-in operational integrations
- Multi-GPU or sharded model deployment

3.3 Ollama

Ollama provides a convenient way to download and run models locally behind an API.

It is particularly useful for:

- Developer laptops
- Local proof-of-concepts
- Offline experimentation
- Small internal deployments
- Testing model compatibility

Ollama's REST endpoints can stream generated output using newline-delimited JSON, and its API responses expose usage and performance information. ([Ollama](#))

Practical rule of thumb

Local development	→ Ollama
Production GPU serving	→ vLLM or TGI
Fastest managed launch	→ Provider API

This is a guideline rather than a hard rule. The final choice depends on model support, hardware, team capability and workload.

4. API-based versus self-hosted

Dimension	Managed API	Self-hosted
Initial setup	Low	High
GPU operations	Provider manages	Your team manages
Scaling	Usually automatic	Must be designed
Model control	Limited	High
Data isolation	Provider-dependent	Greater control
Cost model	Usually token-based	GPU and infrastructure-based
Fine-tuned models	Provider-dependent	Full control
Capacity risk	Rate limits	GPU saturation
Best for	Fast launch, frontier quality	Control, privacy, stable scale

Senior-level decision framework

Do not make the decision using only the cost per token.

Consider:

Total cost = inference cost + engineering cost + GPU idle cost + observability cost

- + operational support cost
- + failure and downtime cost

A managed API can remain the cheaper business choice even when its token price appears higher.

5. Understanding LLM latency

LLM latency should not be treated as one number.

5.1 End-to-end latency

The total time from the client sending a request until the complete response is received.

```
End-to-end latency =  
  network time  
+ queue time  
+ prompt construction  
+ retrieval and reranking  
+ model prefill  
+ token generation  
+ tool execution
```

Track at least:

- p50 latency
- p95 latency
- p99 latency

The average alone can hide serious tail-latency problems.

5.2 Time to first token

TTFT — Time to First Token measures how quickly the user sees the first generated token.

It is heavily affected by:

- Queue waiting time
- Input prompt length
- Retrieval latency
- Model size
- GPU load
- Prompt prefill performance

TTFT is especially important for chat applications because it drives perceived responsiveness.

5.3 Inter-token latency

This is the time between generated tokens after generation begins.

It is also called:

- Time per output token
- Token generation latency

- Decode latency

A response may have good TTFT but generate the remaining tokens slowly.

5.4 Throughput

Two common measurements are:

Request throughput = completed requests / second

Token throughput = generated tokens / second

For LLM serving, token throughput is often more meaningful because one request may produce 20 tokens while another produces 2,000.

6. Performance optimization

6.1 Batching

Batching processes multiple requests together on the GPU.

Without batching:

Request A → GPU
Request B waits
Request C waits

With batching:

[A, B, C] → GPU together

This generally increases GPU utilization and throughput.

Static batching

The server waits until a fixed group of requests is collected.

Useful for:

- Offline summarization
- Embedding generation
- Batch evaluation
- Document processing

Problem:

- Waiting for the batch may increase latency.

Continuous or dynamic batching

Requests can join and leave an active batch as generation progresses.

This is important because LLM requests have different:

- Prompt lengths

- Output lengths
- Stopping conditions

Both vLLM and TGI expose production-oriented batching capabilities. ([Hugging Face](#))

Trade-off

Larger batch → higher throughput → potentially higher queue latency → higher GPU memory consumption
--

A production system usually defines separate limits for:

- Maximum concurrent sequences
- Maximum batch tokens
- Maximum queued requests
- Maximum prompt length

6.2 Prompt and prefix caching

Many requests share the same prefix:

System prompt + company policy + tool definitions + repeated instructions + user-specific content

The repeated portion can potentially be reused rather than recomputed.

Prompt or prefix caching can reduce:

- Prefill computation
- TTFT
- Input-processing cost
- GPU work

A good cache key should include everything that can change the model output:

model version prompt-template version system prompt tools generation parameters tenant or authorization scope relevant context

Never reuse a cached prefix across tenants unless the data is explicitly safe to share.

6.3 Output caching

Output caching returns a previously generated answer for the same request.

Request

↓

Cache lookup

- ├ Hit → return cached response
- └ Miss → call model and cache result

Appropriate cases

- FAQ answers
- Deterministic extraction
- Document summaries that rarely change
- Repeated classification requests
- Low-temperature structured outputs

Risks

- Stale answers
- Cross-user data leakage
- Incorrect reuse when context changed
- Nondeterministic responses being treated as identical
- Old safety policies remaining in cached responses

A safe cache key might be:

```
hash(  
  tenant_id  
  + model_version  
  + prompt_version  
  + normalized_input  
  + document_version  
  + retrieval_config  
  + generation_parameters  
)
```

For RAG applications, document or index version must usually be part of the cache key.

6.4 Semantic caching

Semantic caching retrieves an earlier answer when the new question has a similar embedding.

Example:

```
"Can I cancel my subscription?"  
"How do I terminate my subscription?"
```

This can improve hit rates, but it is riskier than exact caching. Two questions may be semantically close while requiring different answers because of:

- User identity
- Time
- Region
- Account status
- Product version

- Authorization

Use semantic caching mainly for low-risk, stable content and enforce a high similarity threshold.

6.5 Quantization

Models normally store weights in formats such as FP16 or BF16. Quantization represents weights with fewer bits.

16-bit → larger memory requirement
8-bit → lower memory requirement
4-bit → much lower memory requirement

TGI supports multiple quantized model formats, while vLLM also supports serving different quantized model types. ([Hugging Face](#))

Benefits

- Lower GPU-memory requirements
- Larger models may fit on fewer GPUs
- Lower memory bandwidth requirements
- Potentially higher throughput
- Lower infrastructure cost

Risks

- Possible quality degradation
- Numerical instability for some models
- Not every hardware and kernel combination is optimized
- Quantization can sometimes reduce memory without improving latency
- Tool calling, coding and mathematical reasoning may be more sensitive than simple classification

8-bit versus 4-bit

Option	Typical effect
8-bit	Moderate memory reduction, usually lower quality risk
4-bit	Aggressive memory reduction, potentially larger quality impact

Senior-level recommendation

Never approve quantization using only generic benchmarks.

Run your own golden dataset against:

Original model
8-bit model
4-bit model

Compare:

- Task quality
- Safety behaviour
- TTFT
- Tokens per second

- GPU memory
- Cost per request

6.6 Streaming responses

Without streaming:

Generate complete response

↓

Return everything

With streaming:

Generate token

↓

Send token

↓

Generate next token

Streaming usually does not make the model finish faster. It reduces **perceived latency** by showing output earlier.

OpenAI and TGI document SSE-based token streaming, while Ollama supports streamed NDJSON responses. ([OpenAI Platform](#))

Server-Sent Events

SSE is primarily one-way:

Server → Client

Good for:

- Chat text
- Summarization
- Code generation
- Progress events

Example content type:

Content-Type: text/event-stream

WebSocket

WebSocket is bidirectional:

Client ↔ Server

Good for:

- Real-time voice
- Interactive multimodal applications
- Frequent client control messages
- Interruptions and cancellations

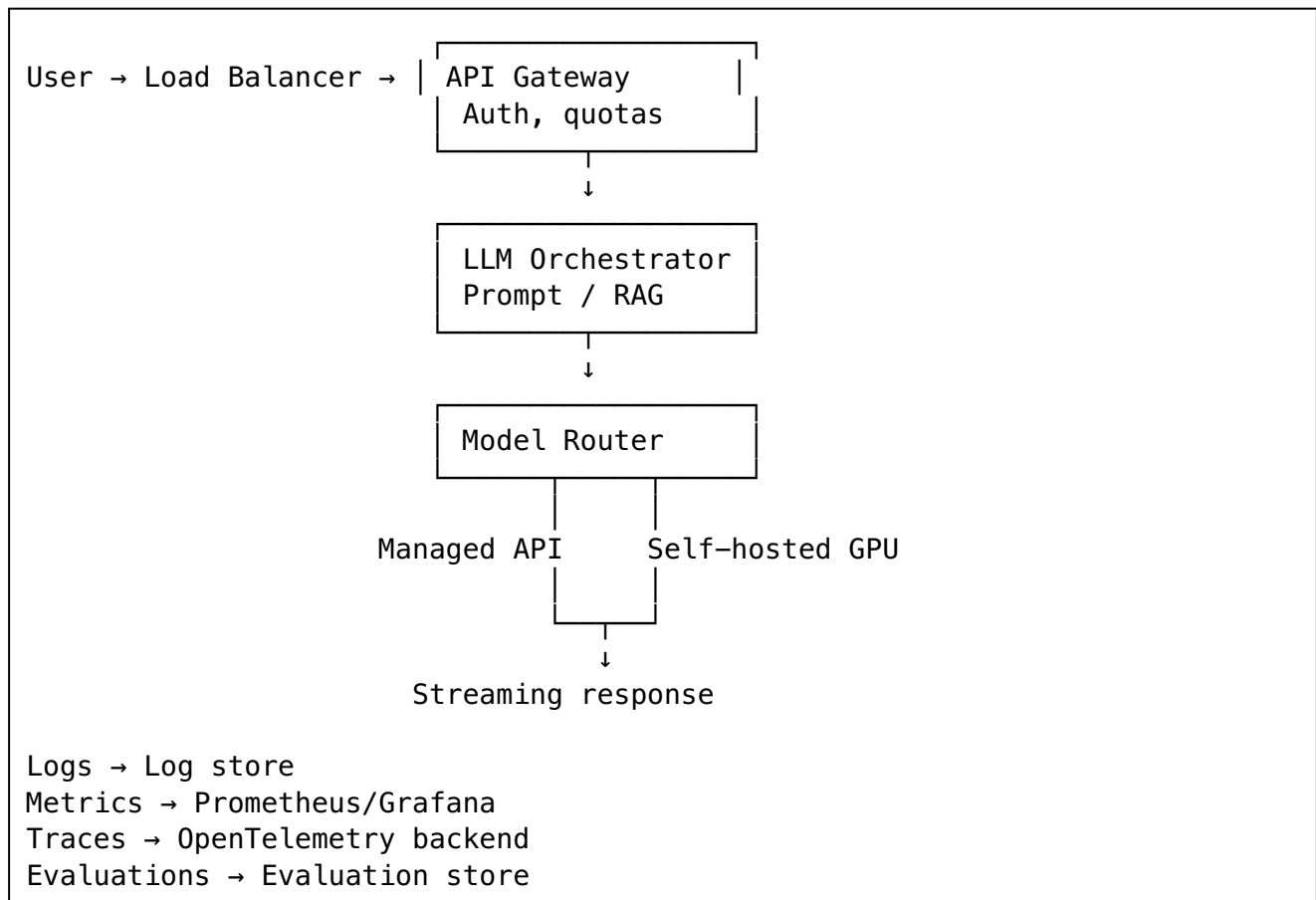
- Continuous agent-status updates

Practical selection

Standard text generation → SSE
Bidirectional real-time interaction → WebSocket
Simple non-interactive API → normal HTTP response

7. Deployment architecture

A production deployment might look like:



8. Docker images and multi-stage Dockerfiles

A multi-stage Docker build separates build dependencies from runtime dependencies.

Docker supports multiple FROM stages and copying only the required artifacts into the final image. This commonly produces a smaller and cleaner runtime image. ([Docker Documentation](#))

Example

```
# Stage 1: install dependencies
FROM python:3.12-slim AS builder

WORKDIR /build

RUN python -m venv /opt/venv
```

```

COPY requirements.txt .

RUN /opt/venv/bin/pip install \
    --no-cache-dir \
    -r requirements.txt

# Stage 2: minimal runtime image
FROM python:3.12-slim AS runtime

ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

COPY --from=builder /opt/venv /opt/venv

WORKDIR /app
COPY app/ /app/

RUN useradd --create-home appuser
USER appuser

EXPOSE 8080

CMD [
    "uvicorn",
    "main:app",
    "--host",
    "0.0.0.0",
    "--port",
    "8080"
]

```

Important production practices

- Pin dependency versions
- Use immutable image tags or image digests
- Run as a non-root user
- Scan images for vulnerabilities
- Keep secrets outside the image
- Add readiness and liveness endpoints
- Do not log API keys
- Keep model weights in a controlled model store or mounted volume when appropriate
- Record the container digest with the deployment version

For GPU images, compatibility among the following must be tested:

```

GPU hardware
CUDA runtime
GPU driver
PyTorch
Inference server

```

9. REST versus gRPC

REST

REST typically uses HTTP with JSON.

```
POST /v1/generate
Content-Type: application/json
```

Advantages:

- Easy to debug
- Browser and client friendly
- Broad ecosystem support
- Natural fit for public APIs
- Works well with SSE

Disadvantages:

- JSON serialization overhead
- Weak contracts unless schemas are enforced
- Less efficient for very high-volume internal communication

gRPC

gRPC commonly uses Protocol Buffers over HTTP/2.

Advantages:

- Strongly typed contracts
- Efficient binary serialization
- Built-in streaming patterns
- Good for internal service-to-service communication
- Client code can be generated

Disadvantages:

- Harder to inspect manually
- Browser support requires additional infrastructure
- Schema changes require discipline

Recommended enterprise pattern

```
External clients → REST/SSE
Internal high-throughput services → gRPC
```

For example, a public FastAPI service may call an internal model worker using gRPC.

10. Canary deployment and rollback

Deploying an LLM is riskier than deploying ordinary deterministic code because a new version may be operationally healthy but behaviorally worse.

A model can:

- Return valid HTTP responses
- Have excellent latency
- Pass health checks
- Still produce less accurate or less safe answers

Canary process

```
Version A receives 100% traffic
      ↓
Deploy Version B
      ↓
Send 1–5% traffic to B
      ↓
Compare quality, safety, latency and cost
      ↓
Gradually increase traffic or rollback
```

Kubernetes supports deployment rollout history and rollback, while canary versions can run alongside existing releases and receive limited live traffic before wider rollout. ([Kubernetes](#))

Canary gates

Monitor both operational and AI-specific metrics.

Operational gates

- Error rate
- p95 and p99 latency
- Timeout rate
- GPU-memory usage
- Queue depth
- Cost per request

Quality gates

- Task success
- Groundedness
- Citation correctness
- Tool-call success
- Refusal correctness
- Human preference
- Complaint rate

Shadow testing

In shadow testing:

```
Real request → Production model → response returned
               ↳ Candidate model → response stored for evaluation
```

The candidate output is not shown to the user.

Advantages:

- Real production distribution
- No user impact
- Direct comparison with the current model

Disadvantages:

- Additional inference cost
- Sensitive data must be handled carefully
- Requires an evaluation pipeline

Rollback unit

For LLM systems, rollback should restore the complete configuration:

```
model
adapter
tokenizer
prompt
retrieval configuration
embedding model
reranker
tools
safety rules
container image
```

Rolling back only the model may not restore the earlier behaviour.

11. LLMOps

LLMOps extends MLOps for systems where behaviour depends on much more than model weights.

A useful configuration identity is:

```
Application behaviour =
  model version
+ prompt version
+ retrieval version
+ tool definitions
+ generation parameters
+ safety policy
+ application code
```

All these components should be versioned.

11.1 Logging prompts and responses

Useful request-level logging fields include:


```
request_id
trace_id
tenant_id or pseudonymous user ID
timestamp
model provider
model version
prompt-template version
input token count
output token count
cached token count
temperature and max tokens
retrieved document IDs
retrieval scores
tool calls
response status
finish reason
TTFT
total latency
error type
estimated cost
user feedback
```

Managed and self-hosted systems expose token and performance-related usage information that can feed these records. OpenTelemetry also defines GenAI-oriented attributes for traces and telemetry. ([OpenAI Platform](#))

Privacy precautions

Logging raw prompts and responses can capture:

- Personal information
- Credentials
- Source code
- Medical or financial data
- Internal documents
- Customer secrets

Safer approaches include:

- Redact PII before logging
- Never log secrets or authentication headers
- Log metadata instead of full content by default
- Encrypt sensitive log stores
- Apply strict access control
- Define retention and deletion periods
- Sample raw conversations rather than retaining all
- Separate production debugging logs from evaluation datasets
- Use tenant-aware authorization
- Record user consent where required

A strong production policy is:

Metadata logging by default Raw-content logging only when justified and protected
--

11.2 Metrics

Performance metrics

- End-to-end latency
- TTFT
- Inter-token latency
- Queue time
- Input tokens per second
- Output tokens per second
- Requests per second
- Batch size
- Cache hit rate

Reliability metrics

- HTTP error rate
- Provider 429 rate
- Timeout rate
- Retry count
- Circuit-breaker activations
- GPU out-of-memory errors
- Stream interruption rate
- Fallback-model rate

Cost metrics

- Input tokens
- Output tokens
- Cached tokens
- Cost per request
- Cost per successful task
- Cost per tenant
- GPU cost per hour
- GPU utilization
- Idle GPU percentage

Quality metrics

- Correctness
- Groundedness
- Hallucination rate
- Tool-call success
- Citation correctness
- Refusal accuracy
- User rating
- Escalation rate

- Task-completion rate

Cost per successful task

This is often better than cost per request:

$$\text{Cost per successful task} = \frac{\text{total inference and infrastructure cost}}{\text{number of successful tasks}}$$

A cheap model that frequently fails may be more expensive overall.

11.3 Distributed tracing

A single user request may involve:

```
API gateway
  → authentication
  → query rewriting
  → embedding model
  → vector database
  → reranker
  → LLM
  → tool call
  → second LLM call
```

Distributed tracing should show the duration and status of every step.

Example:

```
request: 2,400 ms
├── authentication: 20 ms
├── retrieval: 180 ms
├── reranking: 140 ms
├── LLM queue: 250 ms
├── LLM prefill: 310 ms
├── token generation: 1,400 ms
└── post-processing: 100 ms
```

Without traces, teams may blame the model even when the actual bottleneck is retrieval, tool execution or queueing. OpenTelemetry provides conventions for traces, metrics, logs and GenAI operations. ([OpenTelemetry](#))

12. Experiment tracking and model registry

Experiment tracking

Every meaningful experiment should record:

```
base model
model revision
adapter or fine-tuning checkpoint
```

```
quantization
prompt template
dataset version
retrieval settings
generation parameters
evaluation metrics
latency
token usage
cost
code commit
container image
```

MLflow Tracking supports logging parameters, metrics, models and artifacts into organized experiment runs. ([MLflow AI Platform](#))

Example experiments

```
Experiment 1:
Model A + prompt v3 + top-k 5

Experiment 2:
Model A + prompt v4 + top-k 5

Experiment 3:
Model B 4-bit + prompt v4 + top-k 10
```

Experiment tracking prevents decisions such as:

```
| "I think the previous prompt performed better."
```

Instead, the team can reproduce and compare exact configurations.

Model registry

A model registry provides:

- Model versions
- Lineage
- Metadata
- Aliases or deployment labels
- Approval status
- Reproducibility
- Rollback references

MLflow's registry provides centralized lifecycle management, versioning, lineage, tags and aliases. ([MLflow AI Platform](#))

For an LLM, a registry entry should capture more than a weight file:

```
Base model revision
Tokenizer version
Chat template
LoRA adapter
Quantization format
Maximum context configuration
```

```
Inference runtime
Container digest
Evaluation report
Safety approval
```

13. Golden test sets

A **golden test set** is a curated collection of representative and difficult production examples.

Example:

```
{
  "question": "Can an employee access another tenant's report?",
  "expected_behavior": "Refuse and explain access restrictions",
  "category": "authorization"
}
```

A strong golden set contains:

- Common requests
- Edge cases
- Previously reported failures
- Adversarial inputs
- Safety-sensitive inputs
- Long-context examples
- Tool-use examples
- RAG grounding cases
- Multilingual inputs
- Formatting requirements

It should be versioned like source code.

14. Behavioural regression testing

Traditional unit tests often expect an exact output:

```
assert output == "expected value"
```

This works for deterministic operations but is often too strict for natural-language generation.

Better validation types

Exact checks

Useful for:

- JSON schema
- Required fields
- Classification labels
- Tool arguments
- Citation formats

```
assert result["action"] in {"approve", "reject", "review"}
```

Rule-based checks

```
assert account_number not in response
assert response_contains_citation(response)
assert valid_json(response)
```

Semantic evaluation

Compare whether the answer preserves the required meaning rather than matching exact wording.

Pairwise preference

Ask evaluators:

```
For this input, is candidate A or B better?
```

LLM-as-judge

Use a separate model with a rubric to evaluate:

- Correctness
- Groundedness
- Relevance
- Completeness
- Safety

LLM judges should be calibrated against human labels because they can contain bias and inconsistency.

CI/CD regression gate

```
Code or configuration change
    ↓
Run golden dataset
    ↓
Evaluate quality, safety, latency and cost
    ↓
Compare with baseline
    ↓
Pass → deploy canary
Fail → block deployment
```

Example gate:

```
Task success must not drop by more than 2%
Safety failures must not increase
p95 latency must remain below 4 seconds
Average tokens must not increase by more than 10%
JSON-validity rate must remain above 99%
```

15. Recommended production release flow

1. Develop model/prompt/retrieval change
2. Record it as an experiment
3. Run unit and integration tests
4. Run the golden evaluation set
5. Compare quality, latency and cost with baseline
6. Register the approved version
7. Build an immutable Docker image
8. Deploy to staging
9. Run load and failure tests
10. Start shadow or canary traffic
11. Monitor operational and quality metrics
12. Promote gradually or rollback

16. Practical scenario: enterprise RAG assistant

Suppose you are deploying an internal document assistant.

Initial version

- Managed LLM API
- REST service using FastAPI
- SSE streaming
- Vector database
- Exact output caching for approved FAQs
- Prompt and retrieval configuration in Git
- Metadata-only production logging
- Golden dataset of 300 questions

Performance improvements

1. Reduce retrieved chunks from 15 to 6.
2. Add reranking.
3. Cache repeated document retrieval.
4. Stream output immediately.
5. Route simple classification to a smaller model.
6. Limit maximum output tokens.
7. Batch offline document summaries.
8. Add a self-hosted quantized model for predictable internal workloads.

Observability

For each request, record:

```
retrieval latency
LLM TTFT
total latency
input and output tokens
citations returned
model version
```

prompt version document IDs user feedback

Deployment safety

The new model receives 5% of traffic. It is promoted only when:

- Groundedness is not worse than baseline
- Citation accuracy passes the threshold
- p95 latency meets the SLO
- Cost per successful answer is acceptable
- No increase in authorization or PII failures

This demonstrates Senior AI Engineer thinking because it balances model quality with reliability, cost, privacy and operational control.

17. Common pitfalls

Optimizing only average latency

Average latency hides slow tail requests. Track p50, p95 and p99.

Scaling replicas without checking GPU memory

Each replica may require another full copy of the model.

Treating streaming as a throughput improvement

Streaming improves perceived latency but may not reduce completion time.

Caching without tenant isolation

This can expose one customer's response to another customer.

Logging every raw prompt

This creates privacy, compliance and security risk.

Versioning only the model

Prompts, retrieval settings, tools and safety policies also affect behaviour.

Using only generic benchmarks

Production data may differ significantly from public benchmark data.

Canary testing only infrastructure health

An LLM can return HTTP 200 while producing worse answers.

Quantizing without task-specific evaluation

Memory savings do not guarantee acceptable quality.

Autoscaling only on CPU utilization

For LLM serving, queue depth, GPU utilization, active sequences and token throughput are usually more informative.

18. Interview Q&A

1. How would you choose between a managed LLM API and self-hosting?

I would evaluate quality, privacy, traffic predictability, latency, team capability and total operational cost. Managed APIs are usually best for fast launch and bursty demand. Self-hosting becomes attractive when we need model control, strict data isolation, custom weights or predictable high utilization.

2. What is the difference between latency and throughput?

Latency measures how long one request takes. Throughput measures how much total work the system completes, such as requests or tokens per second. Batching may improve throughput while increasing individual request latency.

3. What are TTFT and inter-token latency?

TTFT is the time until the first generated token reaches the user. It includes queueing and prompt-prefill work. Inter-token latency measures how quickly subsequent tokens are produced during decoding.

4. How does continuous batching improve LLM inference?

Continuous batching dynamically combines active requests while allowing completed requests to leave and new requests to join. This handles different prompt and generation lengths more efficiently and improves GPU utilization.

5. Does 4-bit quantization always make inference faster?

No. It primarily reduces memory usage. Speed depends on hardware, kernels, batch size and dequantization overhead. It may also reduce model quality, so it must be evaluated on task-specific data.

6. SSE or WebSocket—which would you use?

I would use SSE for standard one-way token streaming because it is simpler over HTTP. I would use WebSocket when the client and server need continuous bidirectional communication, such as real-time voice, interruption or interactive agent control.

7. What should be logged for an LLM request?

I would log request and trace IDs, model and prompt versions, token usage, generation settings, retrieval metadata, tool calls, latency, errors and user feedback. Raw prompts and responses should be redacted, sampled and retained only under an explicit privacy policy.

8. How would you safely deploy a new LLM version?

First run golden-set and behavioural regression tests. Then use shadow traffic or a small canary percentage. Compare quality, safety, latency, error rate and cost against the baseline. Gradually promote the version or automatically roll back if thresholds fail.

9. Why are traditional unit tests insufficient for LLMs?

LLM outputs are often nondeterministic and multiple answers may be valid. I combine exact schema checks, rule-based assertions, semantic metrics, pairwise evaluation, calibrated LLM judges and targeted human review.

10. What is the most important LLMOps principle?

Version and evaluate the **complete behavioural system**, not only the model. Model weights, prompt templates, retrieval configuration, tools, tokenizer, safety rules and runtime configuration all influence the final response.

Final interview summary

A strong answer can be summarized as:

I would begin with a managed API when speed and model quality are the priorities, while keeping a provider abstraction for portability. For predictable high-scale or sensitive workloads, I would evaluate self-hosting through vLLM or TGI. I would optimize TTFT, token throughput and cost using batching, caching, prompt reduction, model routing and validated quantization. The service would support streaming, distributed tracing and privacy-aware logging. Every model, prompt and retrieval change would pass golden-set regression tests, be registered and versioned, and then be released through shadow or canary traffic with an automated rollback path.